to encrypt n-bit message ... m

KeyGen

p, q = random $n/2$ bit primes

$N = p \cdot q$

$e < N$ s.t. e is relatively prime to $\phi(N)$

relatively prime: $(p-1)(q-1)$

$\boxed{\gcd(e, (p-1)(q-1)) = 1}$

$\boxed{d}$ = multiplicative inverse of e mod $(p-1)(q-1)$

$e \cdot d$ divided by $(p-1)(q-1)$, is 1

publish N, e public

keep d private

Enc(N, e, m):

$c = m^e \bmod N$

return c

Dec(N, d, c):

return $c^d \bmod N$

we think secure because:

- we think factoring $\notin P$
- factoring N, computing $\phi(N)$, find d are all computationally equiv.

GCD

```
gcd(a,b)
    if b=0:
        return a
    else:
        gcd(b, rem(a,b))
```

$a = q \cdot b + r, \quad 0 \le r < b$

rem(252, 105)

$252 = 2 \cdot 105 + \boxed{42}$

$252 = 210 + 42$

ex gcd(252, 105)

gcd(105, 42)

⋮

gcd(21, 0)

so gcd(252, 105) = 21

runtime? $\max(a,b) = N \quad \log_2 N$

n = length N $\quad 2^n \rightarrow \log_2 2^n = n$

```python
p = random_prime(10^20)
q = random_prime(10^20)
print("p =", p)
print("q =", q)
N = p*q
print("N =", p*q)
phi = (p-1)*(q-1)

while True:
    e = random_prime(phi)
    print("try e")
    if gcd(e, phi) == 1:
        break
print("e =", e)

g, x, y = xgcd(e, phi)
d = x % phi
print("d =", d)

m = 31415926535897932384
print("m =", m)

c = power_mod(m, e, N) # m^e % N
print("c =", c)

m_dec = power_mod(c, d, N) # c^d % N
print("decoded =", m_dec)
```